

Raksha

Explainable financial fraud detection and transaction intelligence

Judge briefing | FinTech & Intelligent Financial Systems | 10 September 2026

One-minute pitch

Raksha turns incoming payment activity into explainable risk assessments and investigation cases. It combines configurable checks with account-history statistics, then gives analysts the context to decide what deserves attention. The demo uses simulated INR transactions.

The problem and intended users

Financial operations teams need to identify account takeovers, unusual spending, rapid payments and known risky identifiers without treating every unusual payment as fraud. Banks, payment platforms, wallets and online merchants are potential users; the current implementation serves one shared organization.

What the product demonstrates

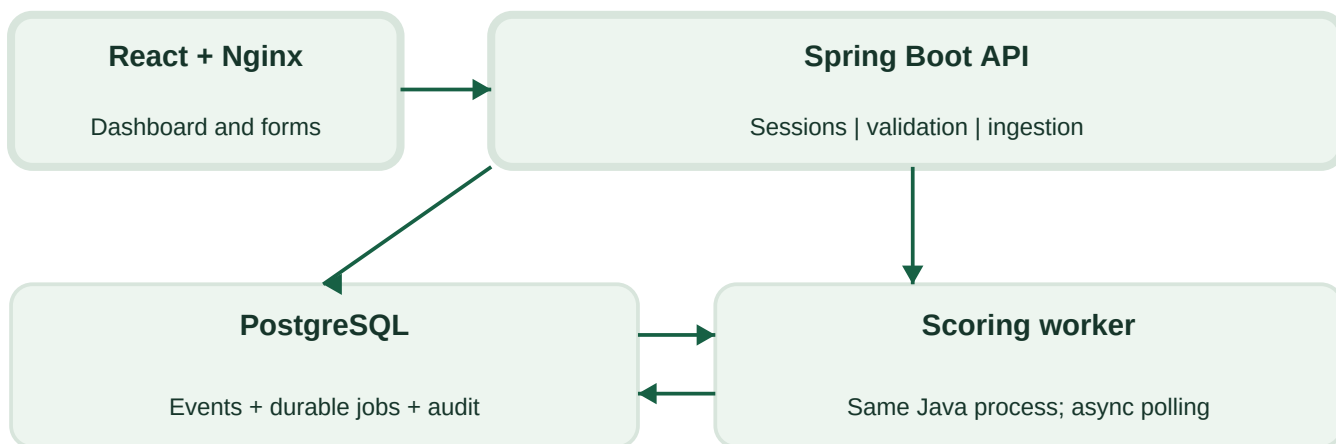
Capability	What a judge can observe
Transaction intake	Create a merchant transaction, then watch its queued assessment appear.
Explainable scoring	See the score, contributing reasons and policy version for each event.
Operational control	Edit points, maintain IP/phone negative lists and pause or resume checks.
Human review	Claim an alert, record evidence and resolve it with a reason.
Financial integrity	Duplicate protection, transactional processing and an audit trail.

Implementation status

Working locally: Java backend, React UI, PostgreSQL, authentication, Docker deployment, rule controls and case workflow. **Prepared:** Kubernetes manifests with separate web/API ingresses. **Not verified:** a public cluster deployment, DNS and TLS.

This brief describes the implemented repository and verified local behavior. It does not claim measured fraud accuracy, production certification or real-money payment processing.

Architecture and transaction flow



Worker reads jobs, persists scores/alerts and completes jobs atomically.

Stack: Java 21 / Spring Boot 3.5; React 19 / TypeScript; PostgreSQL 16; Flyway migrations; Nginx; Docker Compose. Kubernetes resources are included for deployment.

From submission to investigation

Step	Behavior
1. Accept	An authenticated administrator or analyst submits an event. The API validates fields and requires Idempotency-Key to equal eventId.
2. Commit	The transaction and scoring job are stored in one database transaction. The API returns HTTP 202 with its identifier and status.
3. Assess	A scheduled worker locks a ready job and the account, reads earlier history and a consistent rules/policy configuration, and computes points.
4. Record	Score, explanation, feature/rule snapshot, policy version, any alert, audit event and job completion commit together.
5. Review	The UI displays the result. Suspicious/high-risk events become cases for an analyst to claim and resolve.

Why a database queue?

For a 24-hour build, PostgreSQL gives durable work without adding a broker or a database-to-broker dual-write gap. Workers use row locks with SKIP LOCKED; event uniqueness and alert uniqueness protect retries. The default topology is one API/worker instance.

Data model: app_users; transactions; scoring_jobs; detection_rules; risk_policy; alerts; case_notes; audit_events. Amounts are integer paise, and both event time and receipt time are retained.

Eight controllable risk checks

The following are shipped defaults. Administrators can edit each rule's points and enabled state; the policy form controls amount, velocity and classification thresholds. Negative lists begin empty.

Rule	Trigger	Points
Large payment	Amount is at least INR 50,000.	25
Unusual amount	At least 5 prior events; amount / mean is at least 3 and normalized deviation is at least 3.	30
Unfamiliar device	With at least 5 prior events, this device has not appeared before.	15
Unfamiliar country	With at least 5 prior events, this country has not appeared before.	15
Rapid payments	At least 5 events in 5 minutes, including this event.	25
Failed attempts	At least 3 reported preceding failed attempts.	25
Blocked IP	Exact IPv4 or IPv4 CIDR match in the configured negative list.	100
Blocked phone	Exact normalized international phone number in the negative list.	100

Score = min(100, sum of triggered enabled rule points).

Default classes: 0-29 NORMAL; 30-69 SUSPICIOUS; 70-100 HIGH_RISK. Both non-normal classes generate an alert. A score is review priority, not a fraud probability.

What makes the analysis intelligent?

Account-relative statistics distinguish unusual activity from a single fixed limit. Normalized deviation = $(\text{amount} - \text{mean}) / \max(\text{population standard deviation}, \text{mean} \times 0.25)$. The floor reduces unstable scores when historical amounts vary very little. No trained classifier or LLM is currently used.

Worked example

A payment of INR 85,000 from an unfamiliar device and country, with 5 preceding failures and a sufficiently low historical spending baseline, can trigger $25 + 30 + 15 + 15 + 25 = 110$ points, capped at **100 / HIGH_RISK**. The explanation lists each contribution.

Fewer than five prior events disables the behavioral checks and produces an insufficient-history explanation. Other enabled checks still run.

Controls, roles and decisions

Role	Allowed workflow
Administrator	Monitoring and transaction entry; manage rules and policy; review audit history; retry failed jobs; investigate cases.
Analyst	Create transactions and simulations; inspect assessments; claim cases, add notes and resolve assigned cases.
Viewer	Read-only monitoring and investigation visibility; no transaction or rule changes.

Rules & settings

Each rule card shows its active/paused state, description, version and points. IP lists accept IPv4 addresses or CIDR ranges. Phone lists use a plus sign and country code, with spaces, parentheses and hyphens normalized away. Lists are limited to 200 entries.

Saving or pausing a rule uses version checks to reject stale edits, increments the policy version and records an audit event. Changes affect future assessments, including queued events not yet scored. Earlier results remain attached to their original configuration.

Create transaction

The form accepts a merchant name, customer account ID, INR amount, country, device, optional IP/phone and preceding failed-attempt count. Recent merchant names are suggested. The app generates event time and an idempotency ID, then polls the assessment result. This is merchant transaction entry, not merchant onboarding.

Investigation workflow

A reviewer opens the event, reads the risk explanation and claims the case. Only the assigned analyst can resolve it. Resolution requires a reason and either `CONFIRMED_FRAUD` or `FALSE_POSITIVE`. Notes and actions are recorded; version checks prevent stale updates.

Controlled financial decision: a negative-list hit raises risk and routes work to review. Raksha does not transfer funds or automatically decline real payments. Analyst outcomes do not automatically retrain the scoring method.

The UI supports night mode and responsive phone layouts, with transaction entry and investigation details available on smaller screens.

Security, integrity and recovery

Scenario	Implemented behavior / boundary
Duplicate delivery	Unique event ID and request hash return the existing event for identical retries; conflicting content returns HTTP 409.
Failure halfway through scoring	Assessment, alert, audit and job completion share a database transaction. An uncommitted failure rolls these effects back.
Scoring exception	Increasing retry delays; after 3 failures, job becomes DEAD and transaction FAILED. Admin can inspect and retry.
Database outage	Uncommitted work is not accepted as durable; previously committed pending jobs remain available when the database returns.
Delayed/out-of-order event	Event time and receipt time are retained; events over 10 minutes late are marked. Existing scores are not retroactively recalculated.
Concurrent edits	Case, policy and rule versions reject stale writes. Account locks serialize assessment work per account.
API restart	Durable data survives, but in-memory sessions are lost and users sign in again.

Implemented protections

BCrypt password hashes; server-side sessions; HttpOnly and SameSite=Strict cookies; CSRF on mutations; backend roles and case ownership; parameterized SQL and bounded validation; Nginx security headers and CSP. The database is private on the default Compose network.

Limits judges should know

This is one organization with no tenant isolation or MFA. The rate limiter is in memory and groups clients behind the same proxy. Audit rows have no editing API, but database administrators could alter them. IP/phone/device signals are supplied by clients and need trusted provenance in a real integration.

Backups, retention rules, encrypted production secrets, HTTPS, tamper-resistant audit export and tested disaster recovery remain deployment work. A persistent volume is not a backup. Passwords and secrets are intentionally excluded from this briefing.

Deployment, evidence and roadmap

Container and Kubernetes design

Separate images package the Java API and React/Nginx frontend: **adarshbiradar/raksha-api** and **adarshbiradar/raksha-web**. PostgreSQL stores data in a persistent volume. Docker Compose runs the verified local application at localhost:8088.

Prepared ingress	Destination
raksha.rayududev.live	Web service raksha-web:80, prefix /.
api-raksha.rayududev.live	Java service api:8080, prefix /api.

The single raksha-k8s.yml includes namespace, Secret placeholders, configuration, services, deployments, PostgreSQL StatefulSet/PVC and both ingresses. It requires an ingress controller, storage provisioner and reachable images. DNS and TLS require separate configuration. React retains its same-origin /api proxy.

Evidence and hackathon phase mapping

Phase	Demonstrated / remaining
1 - MVP / POC	Functional ingestion, scoring, dashboard, authentication, data model and investigations demonstrated locally.
2 - Productize	Rule controls, roles, merchant entry, explanations and case workflow implemented. Public deployment and billing are not demonstrated.
3 - Productionize	Durable jobs, retries, audit, idempotency, probes and deployment limits implemented/prepared. HA, production security and recovery validation remain.

Verification: Java packaging and React production builds passed. Integration checks covered authentication, CSRF, roles, duplicates, validation, cases, IP/CIDR and phone matching, paused rules, edited points and unchanged prior assessments. Browser checks covered merchant submission, scoring, rule visibility, night mode and phone layout. Kubernetes YAML rendered locally; no live cluster test was completed.

Business direction and scale plan

Proposed model: B2B monitoring SaaS/API priced by transaction volume or enterprise license. This is a business hypothesis; pricing, billing and market demand are not validated. Next priorities are labeled-data evaluation, trusted ingestion and tenant isolation, then shared sessions, indexed account windows, separate workers and measured load testing.

A five-minute judge demonstration

Time	Presenter action and explanation
0:00-0:40	State the problem: distinguish suspicious activity and explain it to an analyst. Open the dashboard and identify transactions, pending work and alerts.
0:40-1:30	As admin, open Rules & settings. Add the synthetic phone +12025550123 to Blocked phone, keep it active at 100 points, and save. Explain versioning and the negative list.
1:30-2:30	Create a new merchant transaction: INR 100, country IN, new account ID, device judge-device, phone +12025550123, zero failed attempts. Wait for HIGH_RISK / 100 and the phone-match explanation.
2:30-3:15	Pause Blocked phone. Create the same kind of payment with a different new account ID. With other default rules and no block-listed IP, expect NORMAL / 0. Show that the previous event stays at 100.
3:15-4:15	Resume the rule. Open the first event in Investigations, claim it, add a note and resolve it as synthetic confirmed fraud with a reason. Show the audit trail.
4:15-5:00	Summarize idempotency, durable jobs and human review. Identify the unverified public deployment and model-evaluation gaps. Restore the test list to its pre-demo contents.

Likely questions and concise answers

Is this AI? It implements explainable statistical anomaly detection plus configurable rules. It does not claim a trained model, an LLM-based decision engine or proven detection accuracy.

How do you reduce false positives? Account-relative baselines, history minimums, adjustable points and analyst outcomes create a tunable workflow. Precision/recall and actual false-positive reduction have not been measured. Evaluation needs labeled data, temporal splits and alert-volume analysis.

Why not Kafka or RabbitMQ? A database queue keeps intake and job creation atomic within the build window. A broker can be added through an outbox when scale and operational needs justify it.

Can it scale today? The shipped topology is one API/worker and one database. Shared sessions, source authentication, benchmarked history queries and tested worker concurrency are prerequisites for stronger claims.

Implementation evidence

Repository sources: README.md; docs/ARCHITECTURE.md; FraudService.java; RiskWorker.java; RuleController.java; SecurityConfig.java; V1/V2 migrations; App.tsx; RuleScreens.tsx; scripts/smoke.mjs; scripts/rules-smoke.mjs; raksha-k8s.yml. Source code takes precedence over older architecture notes where features evolved.